Projekt

Raumvermietung

NoSQL – Datenbanken einsetzen

Modul 165

FS 2026



Inhaltsverzeichnis

1. Ausgangslage 2
2. Anforderungen 3
2.1 Funktionale Anforderungen 3
2.2 Nichtfunktionale Anforderungen 5
3. Dokumentation 6
3.1 Zusatzfunktionen 6
3.2 Visualisierung der Architektur 6
3.3 Technologie-Stack 6
3.4 Testen 6
3.5 Installationsanleitung 6
3.6 Fazit 6
4. Präsentation 7
5. Bewertung 7
6. Projektabgabe 7
6.1 Dokumentation und Applikation 7
6.2 Screencast 7

1. Ausgangslage

Das GIBZ vermietet verschiedene Räumlichkeiten für Veranstaltungen, wie beispielsweise die Aula oder Seminarräume. Informationen zu den verfügbaren Räumen, möglichen Bestuhlungsvarianten, Verfügbarkeit, Preisen sowie ein Catering-Angebot werden auf der offiziellen GIBZ-Webseite zur Raumvermietung bereitgestellt.

Derzeit erfolgt die Reservation über PDF-Formulare, die von den Kunden ausgefüllt und an das Sekretariat per E-Mail übermittelt werden. Benötigt ein Kunde ein Catering, wird dies dem Restaurant *Treff* per E-Mail mitgeteilt. Bei Abweichungen in der Raumausstattung wird der Hausdienst ebenfalls per E-Mail informiert.

Die PDF-Formulare bieten keine automatische Validierung der eingegebenen Daten, wodurch fehlerhafte Angaben auftreten können. Aktuell werden die Formulare ausgedruckt, und die Reservationsplanung erfolgt auf Basis dieser Papierunterlagen.

In der Praxis kommt es häufig vor, dass Kunden Räume reservieren, jedoch noch nicht alle Angaben zu zusätzlicher Ausstattung oder gegebenenfalls zum Catering machen können, etwa weil die genaue Anzahl der Teilnehmer noch nicht feststeht. In solchen Fällen müssen die Mitarbeitenden des GIBZ einige Tage vor dem Termin beim Kunden nachfragen, um die fehlenden Informationen zu ergänzen.

Die Mitarbeitenden sind zunehmend mit einem hohen administrativen Aufwand konfrontiert und suchen nach einer digitalen Lösung, um den Prozess zu automatisieren. Ziel ist es, die Mitarbeitenden zu entlasten und den Reservationsprozess effizienter sowie fehlerfrei zu gestalten.

2. Anforderungen

Die Webseite Raumvermietung sowie die PDF-Formulare sollen durch eine moderne Webapplikation ersetzt werden.

2.1 Funktionale Anforderungen

Informationsbereitstellung

Alle Informationen zu Räumen, Einrichtungsvarianten, Verfügbarkeit der Räume, Preisen, Bildern und AGB sollen für Kunden abrufbar sein.

Raumreservierung

- Kunden können eigenständig Räume reservieren. Zunächst reicht es aus, das Datum, die Start- und Endzeit, den gewünschten Raum sowie die Kontaktdaten anzugeben. Die Reservierung wird anschliessend vom System bestätigt.
- Reservierungen sind nur innerhalb der verfügbaren Zeiten möglich, und der Raum darf zu diesem Zeitpunkt selbstverständlich noch nicht vergeben sein.
- Kunden haben nach der initialen Reservierung bis einige Tage (variabel einstellbar) vor dem Anlass Bearbeitungszugriff auf ihre Reservierung, um Änderungen und Ergänzungen vorzunehmen.

Catering, Einrichtungsvarianten und Material

- Einige Tage vor dem Anlass (variabel einstellbar) müssen die Informationen zu Speisen, Getränken, Einrichtungsvarianten und zusätzlichem Material ausgefüllt sein.
- Die App erinnert Kunden automatisch an fehlende Angaben und ermöglicht das nachträgliche Ergänzen.

Zugriff für Mitarbeitende

- Hausdienst, Sekretariat und Restaurant *Treff* haben Zugriff auf alle Reservierungen und können sie manuell verwalten (CRUD: Create, Read, Update, Delete).
- Daten sollen nach unterschiedlichen Kriterien gefiltert werden können, z. B.:
    - Zeitraum
    - Nutzung von Verpflegung (für Vorbereitung Restaurant *Treff*)
    - Anstehende Termine (zur Vorbereitung des Hausdienstes)
    - Vergangene Termine (für Rechnungsstellung durch das Sekretariat)
    - …

Weiter Anforderung

Für die Note 6 müssen zusätzliche sinnvolle Funktionen definiert und umgesetzt werden.
(Absprache mit Lehrperson)

2.2 Nichtfunktionale Anforderungen

Datenbank

- Die Daten sollen in einer NoSQL-Datenbank gespeichert werden, um eine schnelle und flexible Datensuche zu ermöglichen.
- Regelmässige Backups und Restore-Möglichkeiten müssen sichergestellt werden.
  Der IT-Abteilung soll eine entsprechende Anleitung oder Funktion bereitgestellt werden.
- Der Zugriff auf die Datenbank erfolgt über ein *Data Access Object (DAO)*, um die Datenzugriffe zu kapseln, wiederverwendbar zu machen und die Anwendung von der Datenbank-Implementation zu entkoppeln.

Automatisierung

Erinnerungen an fehlende Angaben (Catering, Einrichtungsvarianten, Material) erfolgen automatisch ohne manuellen Eingriff.

Benutzerfreundlichkeit

- Die App ist webbasiert und intuitiv bedienbar.
- Kunden sollen einfache und klare Eingabemasken für Reservierungen nutzen können.

Zuverlässigkeit / Konsistenz

- Die Integrität der Daten in der NoSQL-Datenbank muss jederzeit gewährleistet sein.
- Benutzereingaben müssen validiert werden, z. B. auf Pflichtfelder, Datumsformate, Zeiträume und zulässige Werte.
- Fehler oder ungültige Eingaben müssen erkannt und dem Benutzer verständlich angezeigt werden.

Flexibilität

- Fristen für Erinnerungen und Deadlines sollen variabel einstellbar sein.
- Anstelle von Hardcoding sollen Konfigurationsparameter, beispielsweise Datenbank-Ports, dynamisch über Parameterübergabe gesetzt werden können.

Entwicklung und Deployment

- Die Entwicklung der Applikation erfolgt über GitLab.
- Die gesamte Applikation soll mittels Docker Compose einfach gestartet und betrieben werden können.

3. Dokumentation

3.1 Zusatzfunktionen

Hier werden alle über die Grundanforderungen hinaus implementierten Funktionen beschrieben, z. B. Erinnerungsfunktionen, Filtermöglichkeiten, erweiterte Benachrichtigungen. Ziel ist es, den Mehrwert der App gegenüber der Basisversion zu zeigen.

3.2 Visualisierung der Architektur

In diesem Abschnitt wird die Systemarchitektur grafisch dargestellt, z. B. als **Komponenten- oder Blockdiagramm**. Es zeigt, wie Frontend, Backend, Datenbank und externe Dienste miteinander verbunden sind. So können Leser das Gesamtsystem schnell erfassen.

3.3 Technologie-Stack

Hier werden alle eingesetzten Technologien und Tools aufgelistet und begründet, z. B.:

- Programmiersprachen (z. B. JavaScript, TypeScript)
- Frameworks (z. B. React, Node.js)
- Datenbank (z. B. MongoDB)

3.4 Testen

Dieser Abschnitt beschreibt die durchgeführten Tests, z. B. Unit-Tests, Integrationstests oder manuelle Tests, sowie deren Ergebnisse. Ziel ist es, die Funktionsfähigkeit, Zuverlässigkeit und Fehlerfreiheit der Applikation nachzuweisen.

3.5 Installationsanleitung

Hier wird Schritt für Schritt erklärt, wie die Applikation installiert und gestartet werden kann, inkl. Konfiguration, Docker Compose, Datenbankanbindung und notwendigen Systemvoraussetzungen.

3.6 Fazit

Im Fazit wird das Projekt kurz zusammengefasst: Erreichte Ziele, Stärken der Applikation, mögliche Verbesserungen und Ausblick auf zukünftige Erweiterungen.

4. Präsentation

Zur Präsentation des Projekts soll ein Screencast erstellt werden, der die Funktionsweise der Applikation live demonstriert. Die Demo soll maximal 10 Minuten dauern und dem Kunden die wichtigsten Funktionen sowie die implementierten Zusatzfunktionen anschaulich präsentieren, um den Nutzen der Applikation zu verdeutlichen. Kreative Ideen zur Gestaltung des Screencasts sind ausdrücklich willkommen und werden positiv bewertet. Es liegt in Ihrer Entscheidung, ob Sie im Screencast selbst als Personen zeigen möchten.

5. Bewertung

Die Projektarbeit fliesst mit **60 %** in die Abschlussnote des Moduls **165 NoSQL-Datenbanken** ein. Die Beiträge der Teammitglieder im GitLab-Repository werden analysiert, weshalb eine aktive Beteiligung aller Mitglieder erforderlich ist. Das Ziel besteht darin, dass die Abschlussnote für alle Teammitglieder identisch ist. Die Bewertung folgt nach der Abgabe des Screencasts.

6. Projektabgabe

6.1 Dokumentation und Applikation

Abgabetermin: 10.06.2026 bis 23:59 Uhr

Speichern Sie den Quellcode (inkl. Dockerdateien) sowie die Dokumentation (Format PDF) im GitLab-Repository und laden Sie den Link zum Repository in der bereitgestellten Gruppen Teams-Aufgabe hoch.

Die Abgabe des GitLab-Links wird als **offizielle Abgabe** des Projekts betrachtet. Das Projekt wird von der Lehrperson erst heruntergeladen und bewertet, wenn der GitLab-Link eingereicht wurde. Vergessen Sie nicht, die Lehrperson als Reporter in Ihr Repository einzuladen. (pascal.bammert)

6.2 Screencast

Abgabetermin: 24.06.2026 bis 23:59 Uhr

Bennen sie Ihre Datei wie folgt:

Screencast_[Klassen].mp4 (z.B.Screencast_INFA2a.mp4)

Laden sie hier ihren Screencast hoch! Pro Team ist nur ein Upload erforderlich.